

Alumnos:

Citlaly Zeferino
Dafne Pineda
Sebastián Erives

Matricula:

T07048221
T07077540
T03004352

Maestro: Adalberto Emmanuel Rojas Perea

Materia: Desarrollo de aplicaciones móviles

Avance del proyecto

1. Introducción

Este documento presenta, de forma clara y ordenada, el desarrollo completo del proyecto realizado para la materia de Desarrollo de Aplicaciones Móviles. El proyecto consiste en una aplicación móvil pensada para la comunidad Rover: un espacio donde los jóvenes puedan dar a conocer sus proyectos sociales, encontrar apoyo de otros compañeros y mantenerse en contacto de forma directa y organizada.

El reporte está dividido en dos grandes partes. La primera describe el avance inicial del proyecto: el problema que se buscó resolver, el público al que está dirigido, las primeras ideas de pantallas y los prototipos visuales. La segunda parte describe el proyecto ya terminado: qué funciones tiene la aplicación, cómo se organiza la información en la base de datos y qué mejoras se hicieron durante el desarrollo.

2. Planteamiento del problema

¿Qué problema resolvemos?

Muchos proyectos sociales creados por jóvenes Rovers no llegan a ser conocidos por otras personas, instituciones u organizaciones. Al no existir un lugar centralizado donde consultarlos, se vuelve difícil que otros jóvenes se enteren de estas iniciativas, las apoyen o se sumen a ellas.

Para atender esta situación, se propuso construir una plataforma móvil cuyo propósito central es reunir en un solo lugar todos los proyectos sociales de la comunidad, junto con un directorio de contacto que permita a los jóvenes comunicarse directamente entre sí. De esta manera se busca:

- Fortalecer el trabajo colaborativo entre los Rovers.
- Dar reconocimiento y visibilidad al esfuerzo de cada desarrollador o equipo de proyecto.

- Ayudar a que las iniciativas sociales crezcan y tengan un mayor impacto dentro de la comunidad.

3. Público objetivo

La aplicación está diseñada principalmente para jóvenes Rovers de entre 18 y 21 años, que forman parte de la red scout y que están interesados en conocer, apoyar o desarrollar proyectos sociales dentro de su comunidad. Este grupo fue elegido porque es la etapa en la que los jóvenes suelen liderar sus propias iniciativas y necesitan herramientas que les faciliten encontrar apoyo y visibilidad.

4. Historias de usuario

Las historias de usuario ayudan a explicar, desde el punto de vista de cada persona que usará la aplicación, qué es lo que necesita y por qué. Esto sirvió como guía para decidir qué pantallas y funciones debía tener el proyecto.

- Como administrador, quiero que la página principal muestre el directorio de contactos, para que la comunicación entre los Rovers sea directa e inmediata.
- Como parte del comité de jóvenes, quiero que los Rovers puedan registrar sus propios proyectos, para que los demás se enteren de ellos y puedan apoyarlos.
- Como desarrollador, quiero poder ver el detalle completo de cada proyecto en una pantalla independiente, para conocer toda la información relevante.
- Como desarrollador, quiero poder consultar la lista completa de proyectos de todos los Rovers, para tener un panorama general de las iniciativas activas.

5. Diseño de pantallas

A partir de las historias de usuario se definieron tres pantallas principales, además de una barra de navegación inferior que permite moverse fácilmente entre ellas.

Directorio de rovers (pantalla principal) — a cargo de Dafne

Muestra el directorio en formato de lista. Cada tarjeta se divide en dos columnas: del lado izquierdo la foto de la persona y del lado derecho su nombre, su cargo (por ejemplo, Rover o Scouter) y sus formas de contacto.

Lista de proyectos — a cargo de Citlaly

Presenta, para cada proyecto, cuatro datos clave: el título del proyecto, el Objetivo de Desarrollo Sostenible (ODS) que atiende, quién es el responsable y cuál es su objetivo principal.

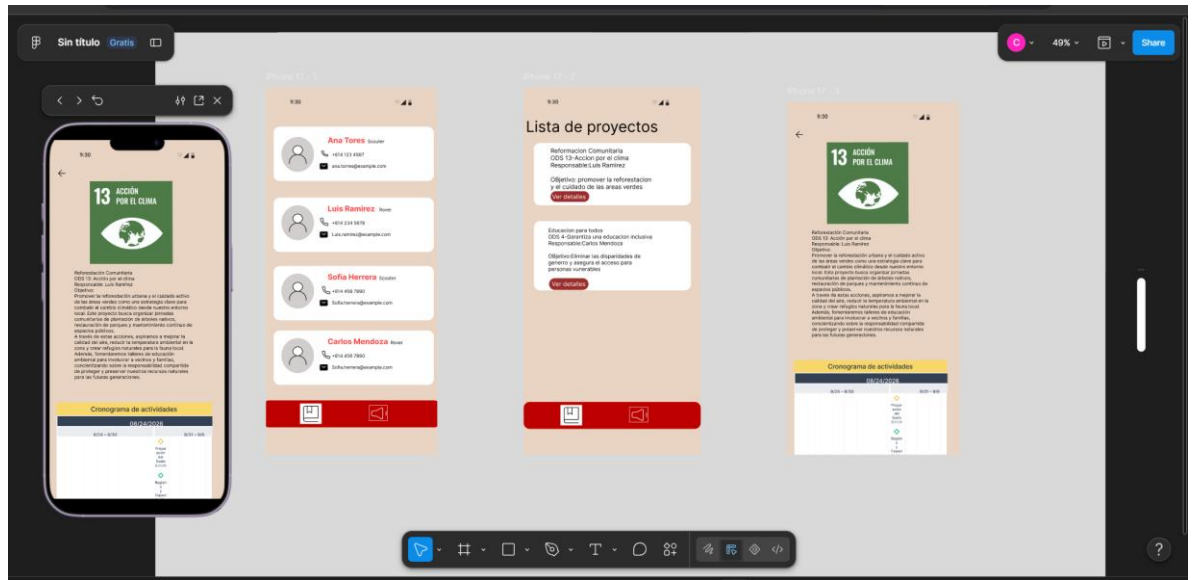
Detalle de proyecto — a cargo de Sebastián

Es la pantalla más completa: incluye un botón para regresar a la lista de proyectos, el título, el ODS relacionado, el objetivo principal, los antecedentes, la justificación, los objetivos específicos, el alcance, la descripción general, los recursos necesarios, el cronograma, los resultados esperados y los datos de contacto del responsable.

Barra de navegación

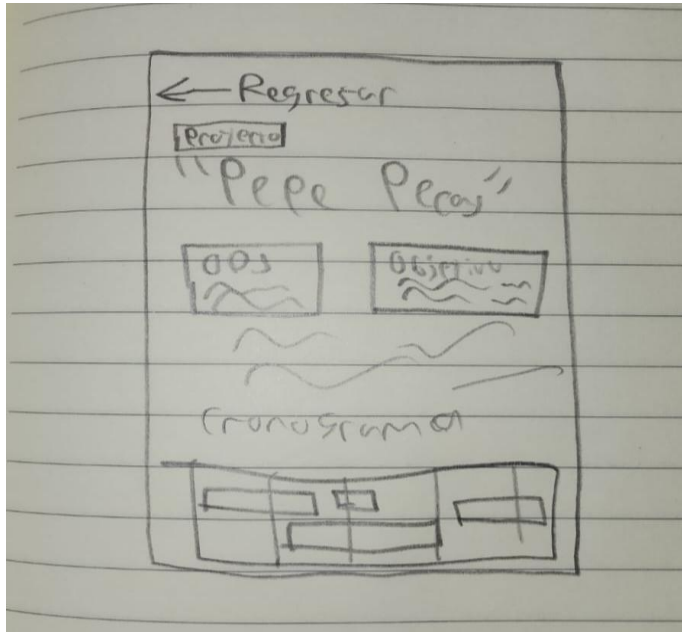
Ubicada en la parte inferior de la pantalla, permite cambiar rápidamente entre el inicio (directorio) y la lista de proyectos.

Estas ideas se plasmaron primero como wireframes (bocetos de baja fidelidad) en Figma, antes de programarlos en la aplicación:



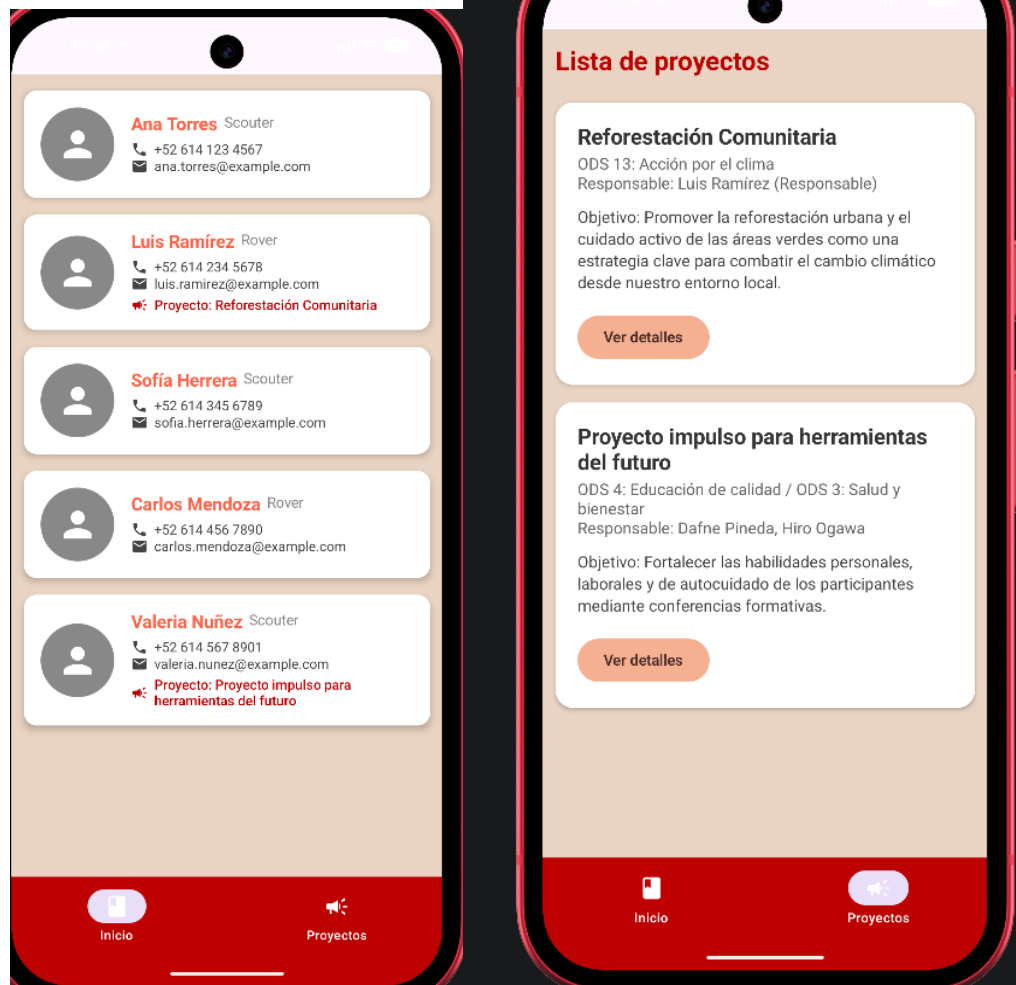
Diseñar wireframes y flujos de usuario en Figma o Adobe XD.

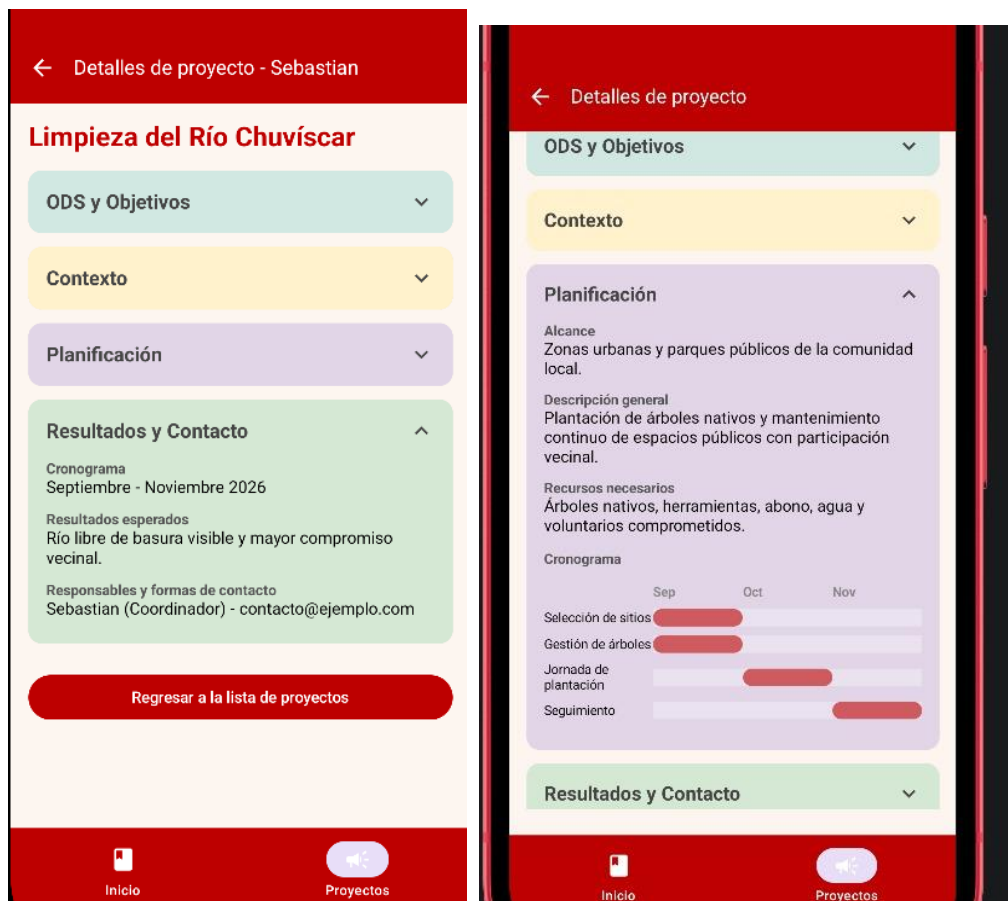




6. Aplicación en el emulador de Android Studio

Una vez definidos los wireframes, las pantallas se construyeron dentro de Android Studio y se probaron en un emulador de dispositivo Android. A continuación se muestran capturas del resultado:





7. Conclusión del avance

En esta primera etapa, la plataforma logró responder a la necesidad de visibilizar y conectar las iniciativas de los jóvenes rovers, transformando proyectos que antes eran individuales en oportunidades reales de contacto y colaboración directa entre la comunidad.

Proyecto final

Nombre de la app: Rover Acción

Esta sección describe cómo quedó terminada la aplicación: los requisitos generales que debía cumplir, todas sus funciones, la organización de la base de datos y las mejoras aplicadas durante las pruebas.

Requisitos del proyecto

Desde el inicio se establecieron cinco condiciones que la aplicación debía cumplir sin importar qué funciones específicas se agregaran después:

- Ser accesible para cualquier usuario.
- Contar con una base de datos que almacene la información de forma ordenada.
- Enviar notificaciones a los usuarios.
- Adaptarse a diferentes tamaños de pantalla y dispositivos.
- Poder generarse como archivo APK y simular su publicación en Google Play.

Funciones de la aplicación

A continuación, se explica, una por una, cada función con la que cuenta la aplicación terminada.

1. Modo oscuro

El usuario puede alternar entre un tema claro y uno oscuro para adaptar la apariencia de la aplicación a su preferencia o a las condiciones de luz.

2. Inicio de sesión (Login)

Los datos de acceso se guardan directamente en el dispositivo del usuario. Así, cada vez que se abre la aplicación, esta revisa si ya existe una sesión guardada; si la encuentra, el usuario entra automáticamente sin tener que volver a escribir sus datos. La información que se solicita es:

- Correo o nombre de usuario

- CUM (clave única del miembro)
- Contraseña

3. Registro (Sign-up)

A diferencia del login, los datos del registro se guardan en la nube, para que el usuario pueda acceder desde cualquier dispositivo. Se solicitan los siguientes datos:

- Nombre completo
- Nombre de usuario
- Correo electrónico
- CUM
- Contraseña
- Confirmación de contraseña

4. Página de inicio

Muestra a todos los usuarios registrados en la aplicación. Al tocar el perfil de alguien más, se puede consultar su información, pero no editarla, ya que esa acción está reservada únicamente para el dueño del perfil.

5. Barra de navegación superior

Incluye dos accesos rápidos: uno para abrir el perfil personal y otro para cerrar la sesión.

6. Pantalla de perfil

Es una de las pantallas más completas de la aplicación:

- El encabezado muestra, del lado izquierdo, una fotografía, y del lado derecho una columna con el nombre completo, el usuario, el correo electrónico y el número telefónico.
- El perfil propio permite editar y eliminar información (gestión CRUD completa); el perfil de otra persona solo puede consultarse.
- Cualquier edición realizada se actualiza automáticamente en la nube.
- Junto a los datos del perfil propio aparece un botón "+" para crear un nuevo proyecto. Al usarlo, se solicitan todos los datos necesarios sin permitir campos en blanco, y hasta abajo hay un botón para

publicarlo; una vez publicado, el proyecto aparece de inmediato en la lista principal.

- Dentro del perfil hay una barra de navegación interna con dos secciones: “Proyectos propios”, donde solo se muestran los proyectos creados por el usuario, y “Voluntariado”, donde aparecen los proyectos en los que el usuario se ha registrado como voluntario.

7. Detalles de proyecto

Además de la información general del proyecto, esta pantalla muestra cuántos participantes se han registrado. Si el proyecto pertenece al usuario, aparecen dos botones adicionales junto al nombre:

- Editar: permite modificar cualquier dato del proyecto; al terminar, un botón hasta abajo guarda los cambios.
- Borrar: elimina el proyecto, pero antes muestra una ventana emergente (Popup) de confirmación para evitar borrados accidentales.

8. Notificaciones

La aplicación avisa al dueño o al organizador de un proyecto que un voluntario se ha registrado a su proyecto.

MainActivity.kt build.gradle.kts (:app) x DataRepository.kt Color.kt Theme.kt colors.xml them: Sync Now Ignore these changes

Gradle files have changed since last project sync. A project sync may be necessary for the IDE to work properly.

```
6      android {
33          buildFeatures {
34              compose = true
35          }
36      }
37
38      dependencies {
39          implementation(platform(dependencyProvider = libs.androidx.compose.bom))
40          implementation(libs.androidx.activity.compose)
41          implementation(libs.androidx.compose.material3)
42          implementation(libs.androidx.compose.material.icons.core)
43          implementation(libs.androidx.compose.material.icons.extended)
44          implementation(libs.androidx.compose.ui)
45          implementation("androidx.compose.material3:material3-window-size-class")
46          implementation(libs.androidx.compose.ui.graphics)
47          implementation(libs.androidx.compose.ui.tooling.preview)
48          implementation(libs.androidx.core.ktx)
49          implementation(libs.androidx.lifecycle.runtime.ktx)
50          testImplementation(libs.junit)
51          androidTestImplementation(platform(dependencyProvider = libs.androidx.compose.bom))
52          androidTestImplementation(libs.androidx.compose.ui.test.junit4)
53          androidTestImplementation(libs.androidx.espresso.core)
54          androidTestImplementation(libs.androidx.junit)
55          debugImplementation(libs.androidx.compose.ui.test.manifest)
56          debugImplementation(libs.androidx.compose.ui.tooling)
57      }
```

royecto > app > build.gradle.kts > dependencies

```

1 Usage
@OptIn(
    ...markerClass = ExperimentalMaterial3Api::class,
    ExperimentalMaterial3WindowSizeClassApi::class
)
@Composable
fun MainScreen(
    currentUser: User,
    cloudRepository: CloudRepository,
    sessionManager: SessionManager,
    isDarkTheme: Boolean,
    onToggleDarkTheme: () -> Unit,
    onLogout: () -> Unit,
    onUserUpdated: (User) -> Unit
) {

    // OBTENER TAMAÑO DE LA VENTANA

    val context = LocalContext.current

    val activity = context as? Activity

    if (activity == null) {
        return
    }

    val windowSize =
        calculateWindowSizeClass(activity)

    val widthSizeClass =
        windowSize.widthSizeClass

```

Correcciones a futuro

Durante las pruebas de la aplicación se identificaron algunos puntos de mejora que quedan planteados como trabajo a futuro:

- Asegurar que el CUM de cada usuario sea único, sin duplicados.
- Agregar una validación que detecte si un CUM ya está dado de alta en la base de datos de ASMAC antes de completar el registro.

- Mejorar la retroalimentación (mensajes de error) cuando los campos de inicio de sesión se llenan incorrectamente.
- Agregar una función de búsqueda por nombre.

Diseño UX/UI y decisiones de diseño

Se cambió el diseño y la paleta de colores del modo oscuro a tonos pasteles, ya que el diseño anterior generaba un contraste visual demasiado fuerte y poco armónico para el usuario.

```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        setContent {
            MyApplicationTheme(darkTheme = isDarkTheme) {
                if (mostrarLogin || currentUser == null) {
                    LoginScreen(
                        cloudRepository = cloudRepository,
                        sessionManager = sessionManager,
                        onLoginSuccess = { user ->
                            currentUser = user
                            mostrarLogin = false
                        }
                    )
                } else {
                    MainScreen(
                        currentUser = currentUser!!,
                        cloudRepository = cloudRepository,
                        sessionManager = sessionManager,
                        isDarkTheme = isDarkTheme,
                        onToggleDarkTheme = { isDarkTheme = !isDarkTheme },
                        onLogout = {
                            sessionManager.clearSession()
                            currentUser = null
                            mostrarLogin = true
                        },
                        onUserUpdated = { updatedUser ->
                            cloudRepository.saveUser(updatedUser)
                            if (updatedUser.id == currentUser?.id) {
                                currentUser = updatedUser
                            }
                        }
                    )
                }
            }
        }
    }
}
```

Ilustraciones en las tarjetas

Ilustraciones en las tarjetas

Para poner ilustraciones utilizamos el componente Canvas, que nos permite dibujar formas propias directamente en las tarjetas.

Cada tarjeta maneja un estilo diferente de ilustración, calcula los tamaños por porcentaje para que así se ajuste sin importar el tamaño de la tarjeta

```
// Ilustración en el lado derecho
if (index % 3 == 1) {
  Canvas(
    modifier = Modifier
      .size(width = 110.dp, height = 90.dp)
      .align(Alignment.BottomEnd)
  ) {
    val path1 = Path().apply {
      moveTo( x = size.width * 0.2f, y = size.height)
      lineTo( x = size.width * 0.65f, y = size.height * 0.25f)
      lineTo( x = size.width * 1.1f, y = size.height)
      close()
    }
    drawPath(path1, color = cardScheme.decorationColor.copy(alpha = 0.5f))

    val path2 = Path().apply {
      moveTo( x = 0f, y = size.height)
      lineTo( x = size.width * 0.45f, y = size.height * 0.45f)
      lineTo( x = size.width * 0.9f, y = size.height)
      close()
    }
    drawPath(path2, color = cardScheme.decorationColor.copy(alpha = 0.8f))
  }
} else {
```

Tecnologías y herramientas utilizadas

Android studio: Para la creación de la aplicación con una zona de prueba local.

Firebase: Base de datos para la aplicación

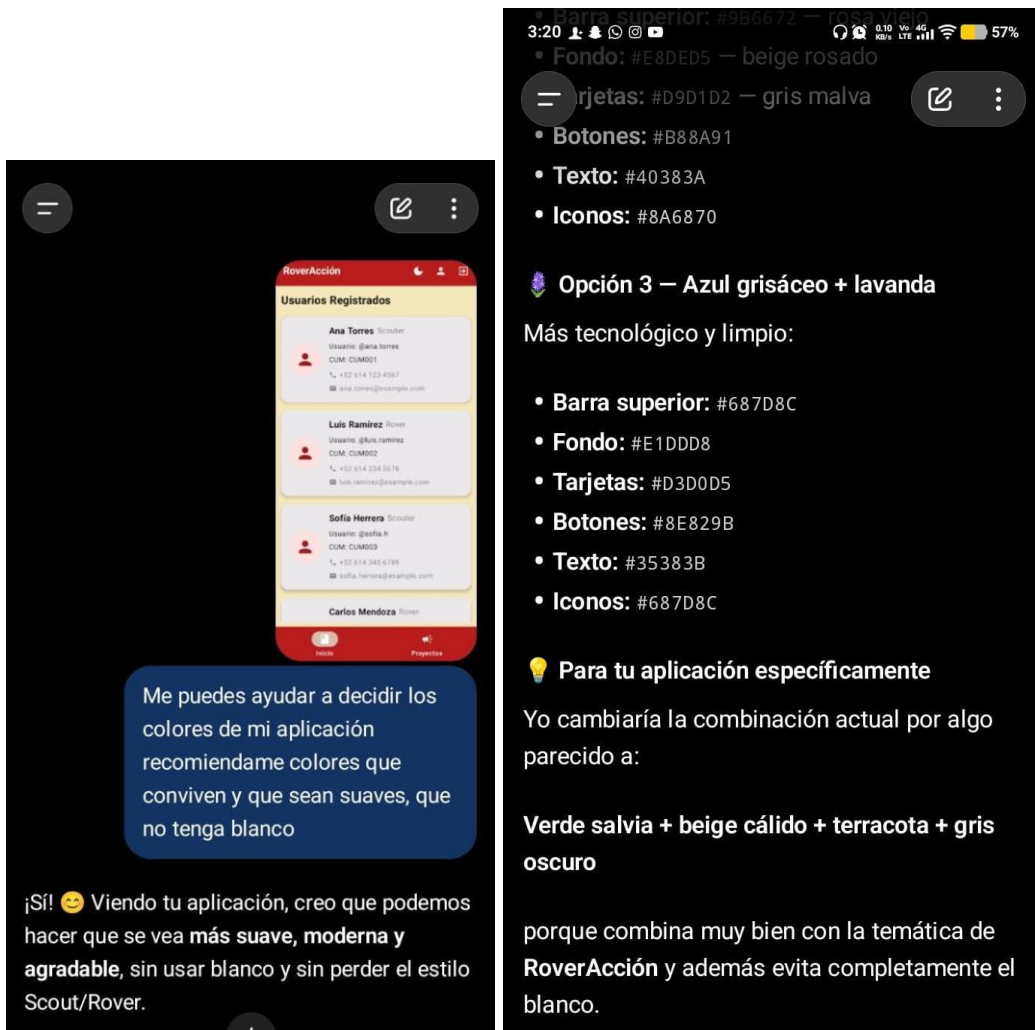
Manejo de datos: Persistencia y la nube

La información de los usuarios y los proyectos viven dentro de la nube para que todos los usuarios tengan acceso a estos en tiempo real. Los únicos datos que son persistentes, son los datos de inicio de sesión del usuario si este desea que se guarden para el inicio de sesión más rápido y cómodo.

Se uso firebase para poder guardar los datos manejados.

Los datos

Ayuda de la IA



The image shows a chat interface on a dark background. On the left, a smaller screenshot of a mobile app named 'RoverAcción' is displayed, showing a list of registered users with red and white cards. Below this, a blue speech bubble contains a request for color recommendations. The main chat area on the right shows the AI's response, which lists color codes for different app elements and suggests a new color palette.

Me puedes ayudar a decidir los colores de mi aplicación recomendame colores que conviven y que sean suaves, que no tenga blanco

¡Sí! 😊 Viendo tu aplicación, creo que podemos hacer que se vea **más suave, moderna y agradable**, sin usar blanco y sin perder el estilo Scout/Rover.

Barra superior: #988672 — rosa viejo
Fondo: #E8DED5 — beige rosado
Tarjetas: #D9D1D2 — gris malva
Botones: #B88A91
Texto: #40383A
Iconos: #8A6870

Opción 3 — Azul grisáceo + lavanda
Más tecnológico y limpio:

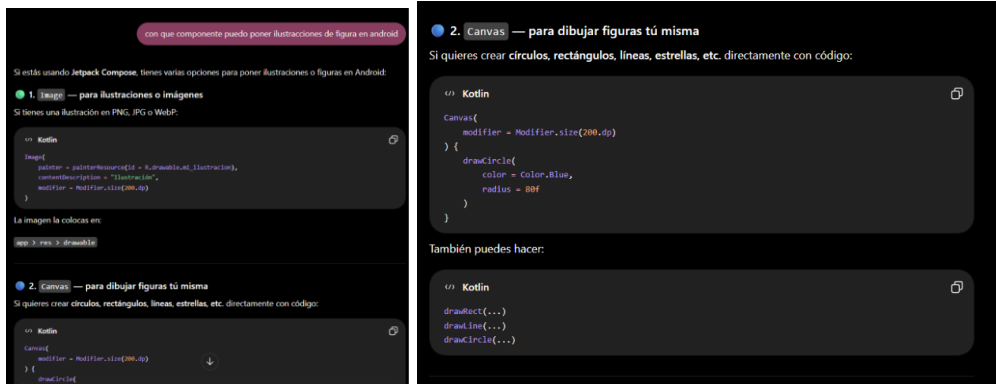
- Barra superior:** #687D8C
- Fondo:** #E1DDD8
- Tarjetas:** #D3D0D5
- Botones:** #8E829B
- Texto:** #35383B
- Iconos:** #687D8C

💡 Para tu aplicación específicamente
Yo cambiaría la combinación actual por algo parecido a:

Verde salvia + beige cálido + terracota + gris oscuro

porque combina muy bien con la temática de **RoverAcción** y además evita completamente el blanco.

Le pedimos recomendaciones a la IA principalmente para elegir una combinación de colores que fuera agradable, moderna y que tuviera relación con la temática de nuestra aplicación.



Utilizamos la inteligencia artificial para resolver dudas sobre cómo implementar figuras e ilustraciones

en Android. Nos ayudó a conocer los componentes que podíamos usar, como Image y Canvas, y nos

mostró ejemplos de código para aplicarlos en nuestro proyecto.

Después de revisar las opciones y comparar la información, elegimos la opción 2 (Canvas), ya que

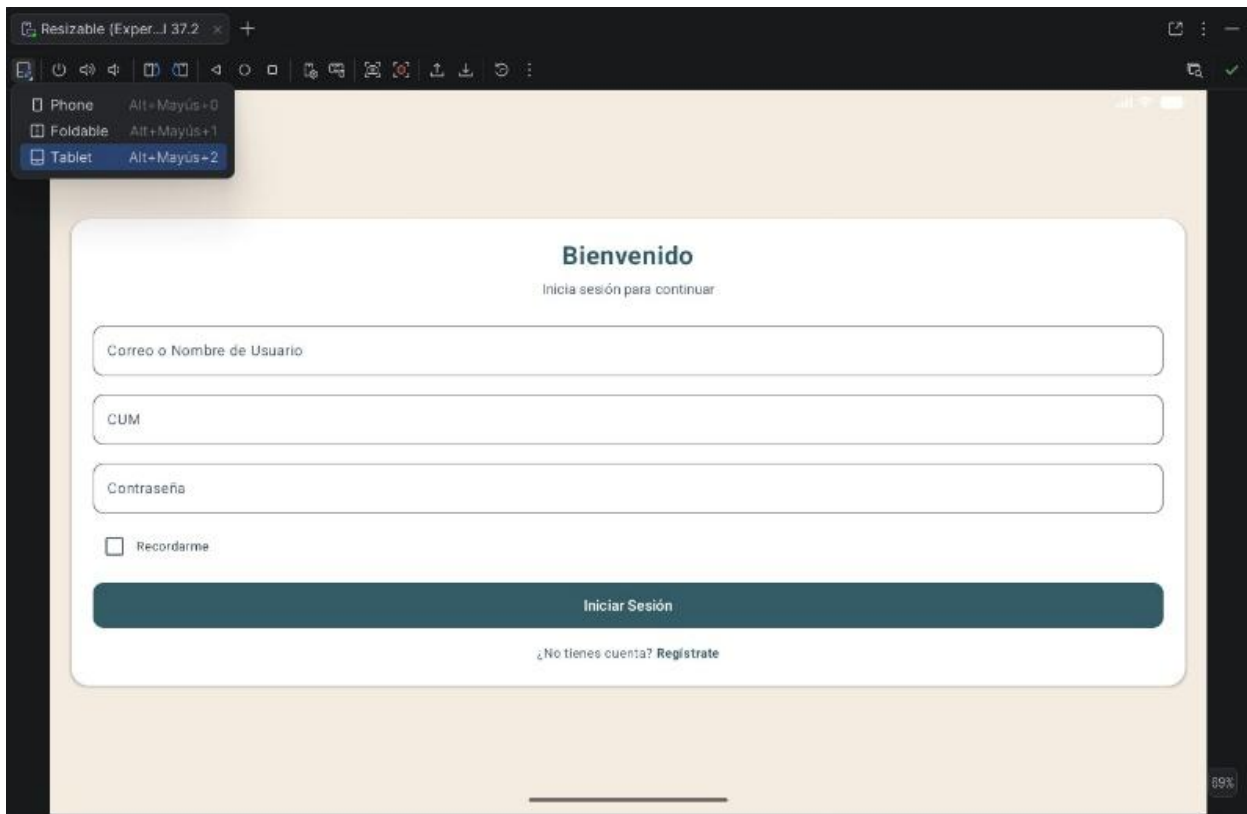
nos permitió dibujar las figuras que necesitábamos de forma personalizada y se adaptaba mejor a

nuestras necesidades en la aplicación.

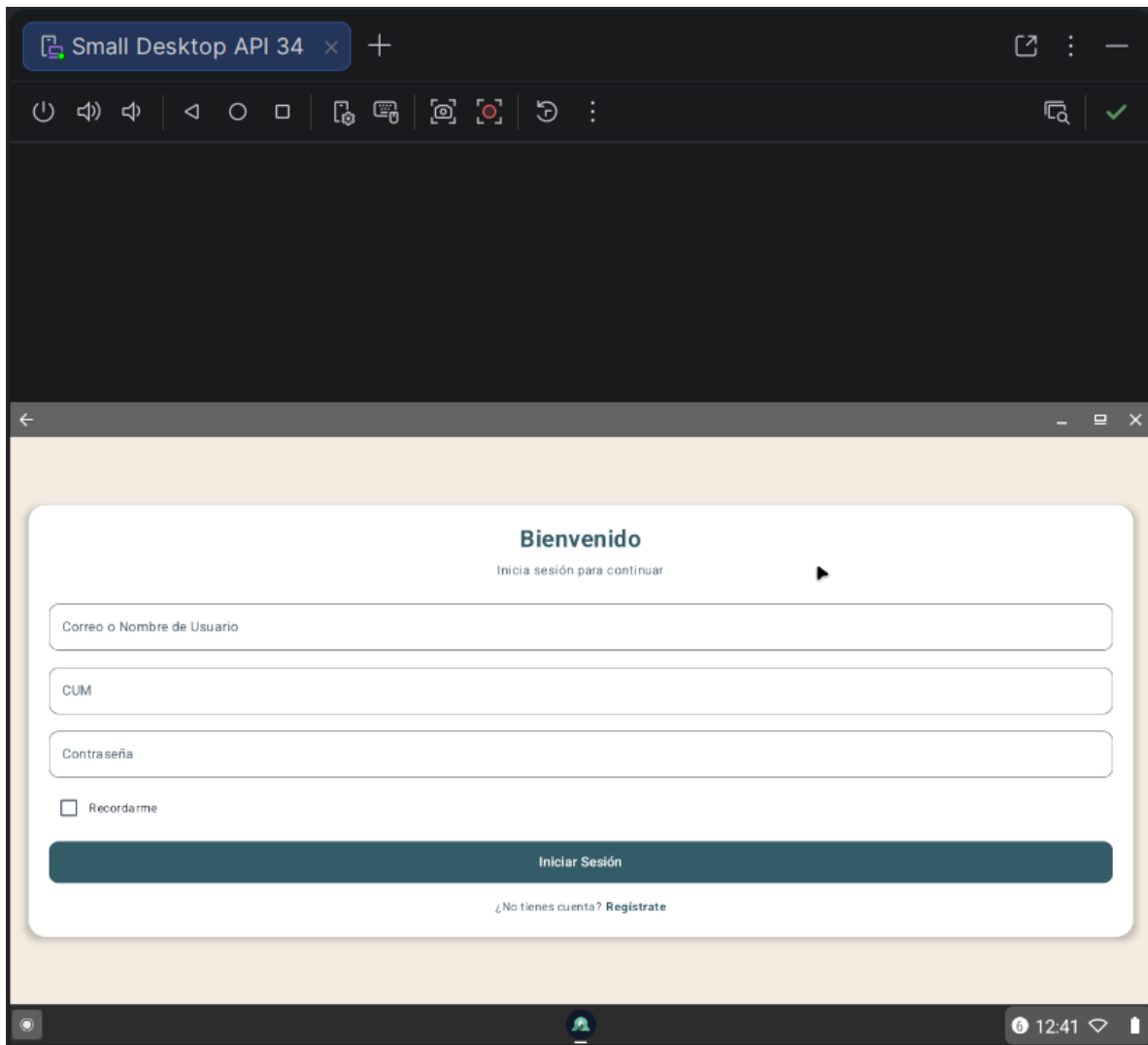
Adaptación a diferentes dispositivos

La motivación principal es que nuestros usuarios puedan tener un seguimiento de los proyectos desde diferentes dispositivos que ofrezcan mayor libertad, el diseño adaptado a cada uno permite que la información se siga viendo.

La interfaz se ajusta automáticamente a diferentes tamaños y modelos de dispositivos, para que la experiencia sea consistente sin importar el equipo desde el que se use la aplicación.



Adaptado a Tablet



Adaptado a Computadora de escritorio

Pruebas

En apoyo con una IA se hicieron las pruebas de la aplicación, algunos de los requisitos que se le comento que considerara fueron:

1. `validarFormulario_datosCorrectos_retornaValido()`:
 - a. Verifica que con todos los datos requeridos completos, formato de correo válido y contraseñas coincidentes, el registro es válido (`esValido = true`).
2. `validarFormulario_camposVacios_retornaErrorGeneral()`:
 - a. Verifica que si falta algún campo obligatorio, se retorne `esValido = false` y el mensaje de error general.
3. `validarFormulario_contraseñasDiferentes_retornaErrorConfirmacion()`:

- a. Verifica que si la contraseña y su confirmación no coinciden, se devuelva "Las contraseñas no coinciden".
4. validarFormulario_correoInvalido_retornaErrorCorreo():
 - a. Verifica que un correo sin formato válido (ej. sin @ o sin dominio) devuelva "Correo electrónico inválido".
5. verificarDuplicados_usuarioExiste_retornaErrorUsuario():
 - a. Simula un usuario existente y verifica que si se intenta registrar el mismo nombre de usuario, se genere el error "Nombre de usuario inválido".
6. verificarDuplicados_correoExiste_retornaErrorCorreo():
 - a. Verifica que si el correo electrónico ya existe en la base de datos, se genere el error "Correo electrónico inválido".
7. verificarDuplicados_cumExiste_retornaErrorCum():
 - a. Verifica que si el CUM ya existe (incluso ignorando mayúsculas/minúsculas), se genere el error "CUM inválido".
8. verificarDuplicados_sinConflictos_retornaValido():
 - a. Verifica que cuando ningún campo está duplicado, la validación sea exitosa.
9. usuarioModel_creacionYPropiedades_correcto():
 - a. Verifica la integridad de la clase de datos Usuario y sus propiedades.

Evidencia con imágenes

Base de datos:

(default) ①	usuarios	KSPfv2oPalt2nCOTtpzE
+ Iniciar colección	+ Agregar documento	+ Iniciar colección
usuarios >	AbdTWoh3KJXFhXeRjAF	+ Agregar campo
	KSPfv2oPalt2nCOTtpzE >	contrasena: "123"
	SyvsoY2o7KElh1QU3WLZ	correo: "123@gmail.com"
	TFibJGyLXgzWZJn7BVXj	cum: "123"
	fAHL0L53PZLMFE1eNliw	nombreCompleto: "123"
	nIkaPezrZLEM4k2S3z6D	telefono: null
	pyB6r7FqGLGQWfv6aKiG	uid: "KSPfv2oPalt2nCOTtpzE"
	wuAdopHCPTZTWHFuDFwc	usuario: "123"

Enlaces

Github <https://github.com/serivesmejia/ActividadesDAM>

Videos https://utmedu-my.sharepoint.com/:f:/g/personal/al07077540_tecmilenio_mx/IgBgxrhKBub7QaZv6GNzBK-xATYRv-ws99PRgN2m1NnmGKI?e=S2bIQA

Landing page:

Conclusión

El proyecto cumplió su objetivo principal: dar visibilidad y conexión a las iniciativas sociales creadas por jóvenes rovers. A lo largo del desarrollo se construyó una aplicación funcional, con inicio de sesión persistente, gestión completa de perfiles y proyectos, notificaciones, una base de datos ordenada en la nube y un diseño